Not Null:

- NOT NULL is an in-line constraint, which specify that a column cannot contain null values.
- It should be defined at column level.

Syntax :
CREATE TABLE table1 (col1 DATATYPE(size) CONSTRAINT cons_name NOT NULL,col2 DATATYPE(size) CONSTRAINT cons_name NOT NULL, col3 DATATYPE(size) , ...);

Example:

CREATE TABLE student (sid NUMBER(4) CONSTRAINT sid_pk PRIMARYKEY, sname VARCHAR2(30) CONSTRAINT name_nn NOT NULL,dob DATE CONSTRAINT dob_nn NOT NULL,gender CHAR(1) CONSTRAINT gndr-chk CHECK (gender ='M' OR gender='F'));

The above example specifies that Student id should not contain any duplicate and null values , student name and date of birth should not contain null values but they can have duplicate values and gender field is restricted with the specified values.

Not Null constraint with Alter statement

Suppose a table has been created without a CHECK constraint and later it was identified to maintain a CHECK constraint in the table , this can be achieved with the help of ALTER statement.

Syntax:

ALTER TABLE table2 MODIFY (col1 DATATYPE(size) NOT NULL);
Example:

CREATE TABLE student (sid NUMBER(4) CONSTRAINT sid_pk PRIMARYKEY, sname VARCHAR2(30) ,dob DATE,gender CHAR(1) CONSTRAINT gndrchk
CHECK (gender ='M' OR gender='F'));

ALTER TABLE student MODIFY(sname VARCHAR2(30) NOT NULL,dob DATE
NOT NULL);

The above statement modifies the existing table schema by adding NOT NULL constraint on table2 using column modify option. The above example specifies that student id should not contain any duplicate and null values, student name and date of birth should always have a value but need not have to be unique and gender field is restricted with the specified values.

Remove Constraint:

In order to remove a NOT NULL constraint which is defined on a table , we need to drop the constraint defined.

Syntax:

ALTER TABLE table1 DROP CONSTRAINT cons_name;

Example:

ALTER TABLE person DROP CONSTRAIN dob_nn;

From the above example we can observe that if at all the government does not want to maintain a NOT NULL constraint for the dob field, that can be removed.

Unique Key:

- A UNIQUE KEY constraint allows to maintain uniqueness of a table column values, depending on the column the constraint is applied.
- A column can have null values though it is defined as a UNIQUE.
- A UNIQUE KEY constraint can consist of one or more columns.
- When required to define multiple columns as UNIQUE KEY , a COMPOSITE UNIQUE constraint can be defined.

Restrictions

- Only one UNIQUE KEY constraint can be defined on a table.
- UNIQUE KEY constraint cannot be defined on the columns having LOB,LONG,LONG RAW,BFILE etc... data types.
- Composite UNIQUE KEY constraint can be defined only as a table level definition.
- A composite UNIQUE KEY constraint cannot have more than 32columns.

Unique with Create statement (Column Level)

A UNIQUE KEY constraint can be defined on a table with the help of a CREATE statement. UNIQUE KEY can be defined immediately after the column definition.

Syntax :
CREATE TABLE table1 (col1 DATATYPE(size) CONSTRAINT cons_name UNIQUE, col2 DATATYPE(size), col3 DATATYPE(size),.....);

Example:

CREATE TABLE supplier (supp_id NUMBER(4) CONSTRAINT supid_unq UNIQUE, name VARCHAR2(20), contact_number NUMBER(15));

The above example specifies that every supplier must have a unique value to identify the supplier.

Unique with Create statement (Table Level)
A UNIQUE KEY constraint can be defined as a table level definition once finishing all the column definitions.

Syntax :

CREATE TABLE table1 (col1 DATATYPE(size), col2 DATATYPE(size), col3 DATATYPE(size), ..., CONSTRAINT cons_name UNIQUE(col1));

Example:

CREATE TABLE supplier(supp_id NUMBER(4), name VARCHAR2(20), contact_number NUMBER(15), CONSTRAINT supid_unq UNIQUE(supp_id));

The above example specifies that every supplier must have a unique value to identify the supplier.

Composite Unique constraint
A UNIQUE KEY constraint can be defined on multiple columns if required. This type of defining on multiple columns is called as composite UNIQUE constraint.

Syntax :

CREATE TABLE table1 (col1 DATATYPE(size), col2 DATATYPE(size), col3 DATATYPE(size), ..., CONSTRAINT cons_name UNIQUE(col1,col2));

Example:

CREATE TABLE warehouse(id NUMBER(6), name VARCHAR2(20),address VARCHAR2(20), contact_number NUMBER(12), CONSTRAINT warehouse_unq UNIQUE(id,name));

The above example specifies that every warehouse need to have a specific id and specific name, which is possible with composite UNIQUE constraint.

Unique with Alter statement

Suppose a table has been created without a UNIQUE KEY constraint and later it was identified to maintain a UNIQUE constraint in the table, this can be achieved with the help of ALTER statement.

Syntax:
ALTER TABLE table2 ADD CONSRAINT cons_name UNIQUE(col1);

Example:

CREATE TABLE new_sim_registration(name VARCHAR2(20), gender CHAR(1), dob DATE, existing_contact_no NUMBER(12), new_sim_no NUMBER(12), address_proof VARCHAR2(10));
ALTER TABLE new_sim_registration ADD CONSTRAINT newsim_unq UNIQUE(new_sim_no);

The above example specifies that, whenever a customer wants to take a new simcard , the telephone service provider will collect all the details of the customer and makes the new_sim_no as UNIQUE so that the new_sim_no can be filled later once the address_proof submitted is verified.

Remove Constraint:

In order to remove a UNIQUE KEY constraint that is defined on a table, we need to drop the constraint defined.

Syntax:

ALTER TABLE table1 DROP CONSTRAINT cons_name;

Example:

ALTER TABLE location DROP CONSTRAINT newsim_unq;

From the above example, we can observe that if at all the government does not want to maintain a UNIQUE constraint, which can be removed.